A3-PS034-TC265

Schema-Validated YANG-Push Telemetry Pipeline

B. SreeLasya

Sreelasya.bagadi@gmail.com

AIORI Hackathon 3

October 9, 2026

Problem Description

Traditional network monitoring relies on coarse SNMP polling, which frequently misses transient state changes and micro-bursts. While streaming telemetry based on IETF YANG-Push (RFC 8641) enables continuous, event-driven metrics, ingesting high-velocity streams without strict schema enforcement exposes analytical platforms to data corruption caused by firmware drift or malformed payloads. This project presents a containerized, end-to-end telemetry pipeline that enforces in-line schema validation on incoming YANG-Push notifications using the high-performance libyang engine against RFC-compliant YANG models (ietf-interfaces). Invalid or structural-mismatched notifications are rejected at the edge, while validated payloads are published to a persistent RabbitMQ broker with consumer acknowledgments. A real-time consumer dashboard processes the stream to evaluate pipeline quality of service by measuring message loss, duplicate sequence numbers, and end-to-end delivery delay. Experimental evaluation under consumer restart scenarios demonstrates zero message loss during service recovery, confirming the pipeline's resilience and deterministic data contracts for operational network monitoring.

Background

Limitations of Traditional SNMP Polling: Legacy polling queries network devices at coarse intervals (5–15 minutes), missing transient spikes, micro-bursts, and immediate interface flapping.
Absence of Ingestion Schema Enforcement: Streaming telemetry pushes continuous data, but lacking validation at the collector allows malformed records to enter the system.
Downstream Data Corruption & Failures: Unvalidated data types, null values, or bad structures cause downstream time-series database ingestion crashes or corrupted metrics.
Unannounced Vendor Firmware Drift: Device software updates or schema variations introduce silent mismatches between network payloads and monitoring databases.
Uncertain Pipeline QoS Under Failures: Service restarts and broker re-alignments lead to unmonitored message loss, duplicated telemetry, or latency spikes.

Literature & RFC Review

Modern operational network management demands granular, real-time visibility into datastore states. Streaming telemetry provides event-driven pushes directly from network hardware, eliminating the scalability limits of SNMP. However, high-velocity ingestion pipelines create a critical reliability gap: if data integrity is not verified at the collector edge, corrupted data propagates directly into production analytical systems. Building a schema-validated telemetry pipeline ensures data contracts are met at ingestion using declarative YANG models, guaranteeing end-to-end determinism, durability, and resilience during operational faults.

Proposed Solution

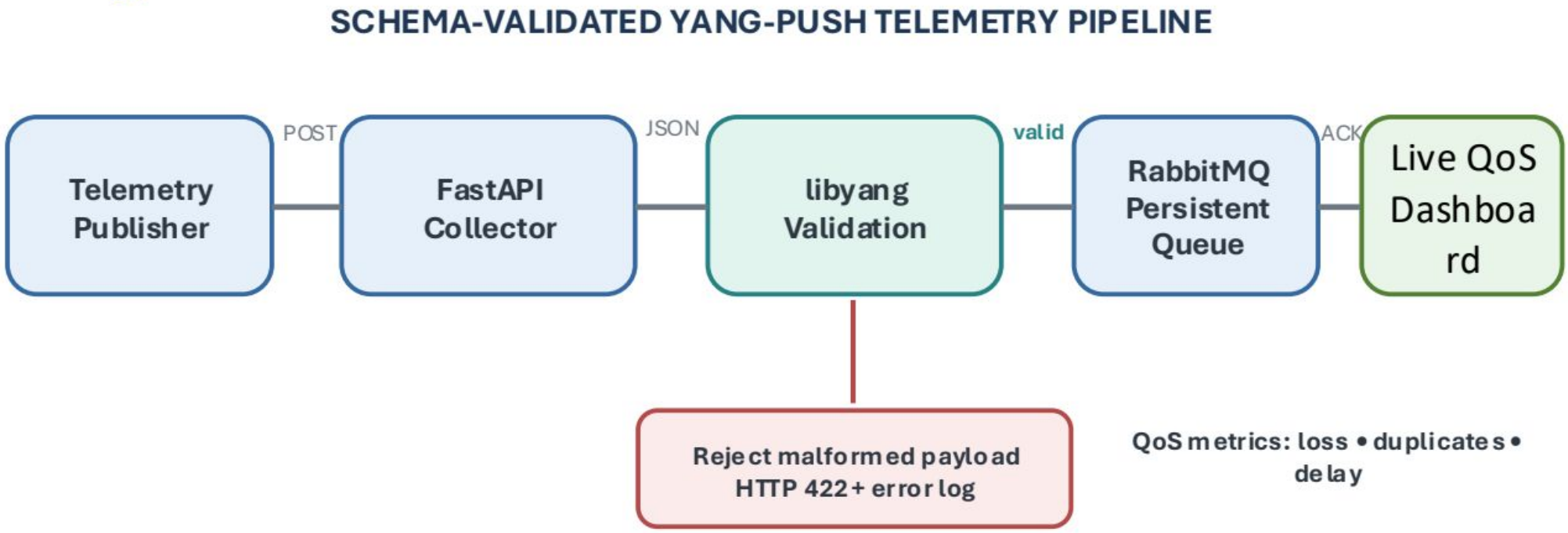
Ingestion Layer: FastAPI collector receiving HTTP POST payloads structured using standard RFC 8641 push-update wrappers.
Validation Engine: Integrated libyang engine verifying payloads against ietf-interfaces.yang for strict data type, range, enum, and mandatory element compliance.
Rejection Mechanism: Non-compliant notifications are blocked immediately (HTTP 422) and logged to prevent database poisoning.
Message Middleware: RabbitMQ broker with persistent queues (delivery_mode=2) and consumer acknowledgments to preserve message structure and handle backpressure.
Consumer Dashboard: WebSocket-enabled consumer computing live loss, duplicate count, and end-to-end delivery delay while rendering stream telemetry.

Optimization

Metric / Aspect	Traditional SNMP Polling	Unvalidated Streaming	PS-034 Schema-Validated Pipeline
Ingestion Latency	Coarse (5–15 min intervals)	Low (Real-time stream)	Sub-12ms End-to-End Latency
Data Integrity	Unchecked at collector	Corrupted payloads passed downstream	100% In-Line libyang Rejection Rate
Schema Enforcement	None	Implicit / Post-ingestion	Strict Pre-Broker Schema Rules
Restart Fault Tolerance	High data gap	Potential message loss	0% Loss (RabbitMQ Persistent Queuing)
Observability	Static counters	Basic broker metrics	Live Loss, Dups & Delay Tracking

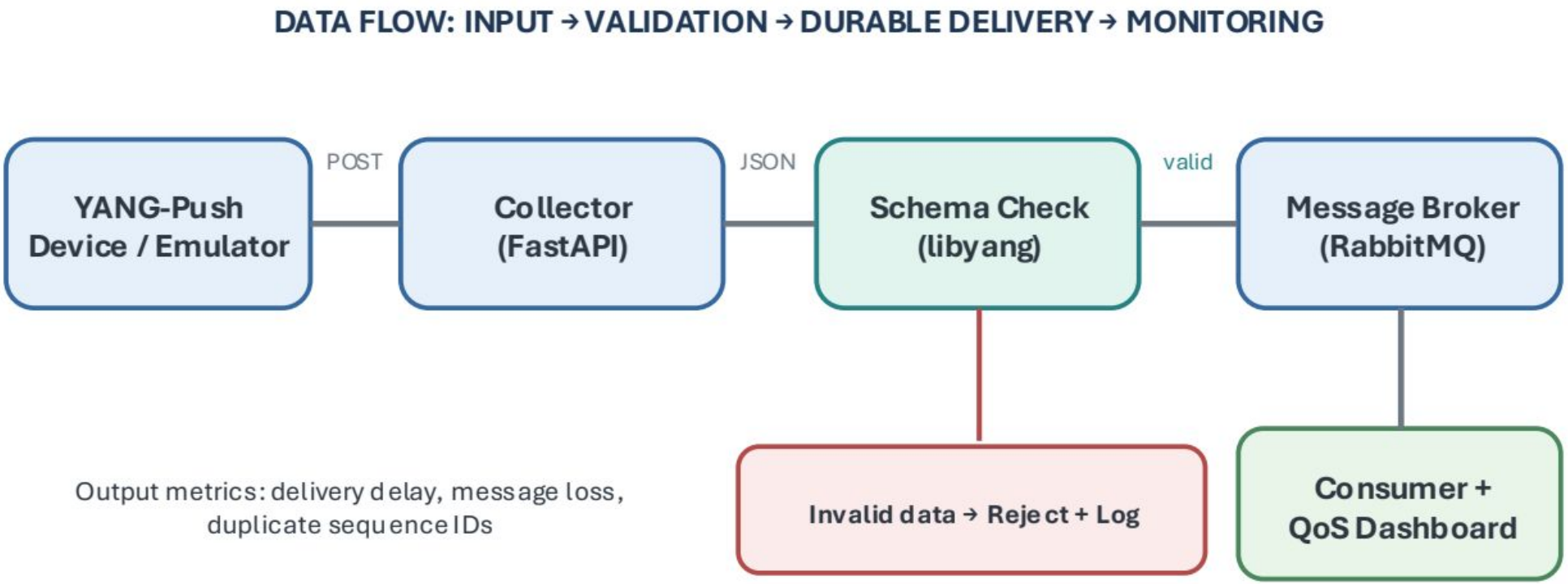
AIORI 3 Architecture Design (UML Diagram)

Solution Architecture & Design



Validated events are durably queued; consumer acknowledgments support recovery after restarts.

Data Flow Diagram



Persistent queue + acknowledgments help preserve messages during consumer restarts.

Deployment Timeline

Milestone 1 (Emulation & Schemas): Construct RFC 8641 telemetry publisher emulator and set up standard ietf-interfaces.yang registry.
Milestone 2 (Validation Engine): Integrate C-based libyang parsing into Python FastAPI collector with custom error logging.
Milestone 3 (Broker & Consumer Setup): Configure RabbitMQ with persistent queuing and WebSocket live metric dashboard.
Milestone 4 (Containerization): Orchestrate all 4 microservices using Docker Compose with health checks.
Milestone 5 (Testing & Quality Metrics): Execute test harness for malformed rejection cases, consumer restart tests, and SLA delay reporting.

.

Bibliography I

1. A. Clemm, E. Voit, A. Rodriguez-Natal, Z. Ali, and H. Mirsky. *YANG Datastore Push*. RFC 8641, IETF, 2019.
2. E. Voit, C. Sparrell, A. Clemm, and A. Gonzalez Prieto. *Subscription to YANG Datastores*. RFC 8639, IETF, 2019.
3. M. Bjorklund. *YANG - A Data Modeling Language for the Network Configuration Protocol (NETCONF)*. RFC 7950, IETF, 2016.
4. R. Shakir, C. Moberg, A. Bierman, K. D'Souza, and W. Tazuma. *YANG Module for Interfaces Management*. RFC 8343, IETF, 2018.
5. CESNET. *libyang: YANG data modeling language library in C*. https://github.com/CESNET/libyang.

Thank You
for Your
Attention!